

runtime optimisations, Velox allows organisations to slash query latencies, eliminate JVM garbage collection overhead, and achieve up to a 3x reduction in hardware footprints.

The future of unified execution

Velox represents a fundamental paradigm shift in database infrastructure. By extracting common execution primitives, vectorized data layouts, expression compilers, memory pools, and file readers into a single, highly optimised C++ library, Velox delivers on three key promises:

- **Unmatched efficiency:** Slashes query latencies, eliminates JVM garbage collection pauses, and delivers up to a 3x reduction in cluster server counts.

- **Semantic consistency:** Guarantees that SQL functions produce the exact same results whether executed in Spark SQL, Presto C++, or PyTorch.

- **Hardware acceleration ready:** Seamlessly scales from x86 and Graviton4 CPUs to NVIDIA GH200 Grace Hopper GPUs via cuDF. 

References

- GitHub repository, <https://github.com/facebookincubator/velox>
- Official documentation, <https://facebookincubator.github.io/velox/>
- NVIDIA blog, <https://developer.nvidia.com/blog/accelerating-large-scale-data-analytics-with-gpu-native-velox-and-nvidia-cudf/>
- Apache Spark with Gluten+Velox benchmarks, <https://awslabs.github.io/data-on-eks/docs/benchmarks/spark-gluten-velox-benchmark>

By: Saurabh Mahawar

The author is a developer relations engineer at IBM. He has built and nurtured developer communities, organised large scale events, and led strategic initiatives to drive developer engagement and adoption of technology.

Acknowledgment: The author would like to thank Naveen Kumar Mahadevuni, software engineer, IBM, for his support in writing and reviewing this article.

■ ■ Continue from page...66

PQC tools

Tool/Library	Role	Notes
liboqs	Reference C toolkit	Broad algorithm coverage
OpenSSL 3.2+	TLS/SSL provider	ML-KEM and ML-DSA support
BouncyCastle	Java / C#	Wide PQC support
pq-crystals.org	Reference implementations	ML-KEM and ML-DSA
pqc-forum	Community discussion	Standards updates
Go/Rust/Python/JS bindings	Language support	Rapidly maturing

- is not just for cryptographers — operators and developers need guidance too.

A deadline, not an option

The PQC transition is not optional — it is a deadline. Attackers are already harvesting encrypted traffic. When quantum machines arrive, only projects that migrated early will remain secure.

Open source must lead. Audit dependencies, adopt PQC libraries, build crypto agility, and engage in standards. The projects that act now will define the secure infrastructure of the quantum era. Those that wait risk leaving their users exposed. 

References

- NIST, 'Announcing Draft Federal Information Processing Standards (FIPS) for Post-Quantum Cryptography', 2023–2024
- Open Quantum Safe Project, <https://openquantumsafe.org/>
- IETF, 'Hybrid Post-Quantum Key Encapsulation' and related TLS/SSH drafts
- Bernstein, D.J. and Lange, T., 'Post-Quantum Cryptography', <https://pqcrypto.org/>
- ENISA, 'Post-Quantum Cryptography: Current state and quantum mitigation'
- Google Chrome Enterprise, 'PQC and the need for crypto agility', <https://security.googleblog.com/>
- Cloudflare, 'The future of PQC in TLS: A first look at ML-KEM', <https://blog.cloudflare.com/>
- PQShield, 'Post-Quantum Cryptography: State of the Art and Practical Migration'
- Trail of Bits, 'PQC migration in open source software', <https://blog.trailofbits.com/>
- SANS Institute, 'Preparing for the post-quantum cryptographic transition'
- Mozilla NSS and OpenSSL PQC Roadmaps

By: Kousik Maiti

The author has over 18+ years of industry experience. His areas of interest include Linux systems administration, security and forensics.